

Docker: Explicación Detallada del Dockerfile y docker-compose.yml

Guía completa para entender la configuración Docker del proyecto Ecodisseny

Índice

-  Conceptos Básicos
-  Dockerfile Explicado
-  docker-compose.yml Explicado
-  Volúmenes y Persistencia
-  Redes y Comunicación
-  Variables de Entorno
-  Comandos Útiles

Conceptos Básicos

¿Qué es Docker?

Docker es una plataforma que permite empaquetar aplicaciones y sus dependencias en **contenedores** ligeros y portables.

¿Por qué usar Docker?

 Desarrollo Local	→	 Producción VPS
└─ Python 3.12		└─ Python 3.12 ✓
└─ PostgreSQL 15		└─ PostgreSQL 15 ✓
└─ Django 5.2.4		└─ Django 5.2.4 ✓
└─ Dependencias exactas		└─ Dependencias exactas ✓

Sin Docker: "Funciona en mi máquina" 😞 **Con Docker:** "Funciona en todas las máquinas" 🎉

Arquitectura del Proyecto

 Docker Containers
└─  Web Container (Django)
└─ Puerto: 8000
└─ Volumen: código fuente
└─ Volumen: archivos estáticos/media
└─  DB Container (PostgreSQL)
└─ Puerto: 5432 (interno)
└─ Volumen: datos de BD
└─ Variables: usuario/contraseña

Dockerfile Explicado

El **Dockerfile** define cómo construir la imagen de tu aplicación Django.

Estructura del Dockerfile

```
# =====
# ETAPA 1: Imagen base
# =====
FROM python:3.12-slim
```

¿Por qué python:3.12-slim?

- ✓ **python:3.12**: Versión específica de Python
- ✓ **slim**: Imagen ligera (menos tamaño, más rápida)
- ✓ **Debian-based**: Compatible y estable

Configuración del Sistema

```
# =====
# ETAPA 2: Configuración del sistema
# =====
ENV PYTHONUNBUFFERED=1
ENV PYTHONDONTWRITEBYTECODE=1
```

Explicación de variables:

Variable	Función	¿Por qué?
PYTHONUNBUFFERED=1	Desactiva buffering de Python	Vemos logs en tiempo real
PYTHONDONTWRITEBYTECODE=1	No crea archivos .pyc	Contenedor más limpio

Instalación de Dependencias del Sistema

```
# =====
# ETAPA 3: Dependencias del sistema
# =====
RUN apt-get update && apt-get install -y \
    build-essential \
    libpq-dev \
    curl \
    && rm -rf /var/lib/apt/lists/*
```

¿Qué instala cada paquete?

Paquete	Función
<code>build-essential</code>	Compiladores (gcc, make) para instalar paquetes Python con extensiones C
<code>libpq-dev</code>	Headers de PostgreSQL para psycopg2
<code>curl</code>	Herramienta para descargas HTTP
<code>rm -rf /var/lib/apt/lists/*</code>	Limpia cache de apt (reduce tamaño de imagen)

Directorio de Trabajo

```
# =====  
# ETAPA 4: Directorio de trabajo  
# =====  
WORKDIR /app
```

¿Qué hace?

- Crea el directorio `/app` dentro del contenedor
- Establece `/app` como directorio actual
- Todos los comandos siguientes se ejecutan desde `/app`

Instalación de Dependencias Python

```
# =====  
# ETAPA 5: Dependencias Python  
# =====  
COPY requirements.txt .  
RUN pip install --no-cache-dir -r requirements.txt
```

Proceso paso a paso:

1. `COPY requirements.txt .`:
 - Copia `requirements.txt` desde tu máquina al contenedor
 - El `.` significa "directorio actual" (`/app`)
2. `pip install --no-cache-dir -r requirements.txt`:
 - Instala todas las dependencias Python
 - `--no-cache-dir`: No guarda cache (imagen más pequeña)
 - `-r requirements.txt`: Lee dependencias del archivo

¿Por qué copiar requirements.txt primero?

- 🚀 **Cache de Docker**: Si `requirements.txt` no cambia, Docker reutiliza esta capa

- ✂ **Builds más rápidos:** Solo reinstala dependencias si cambian

Copia del Código

```
# =====  
# ETAPA 6: Código de la aplicación  
# =====  
COPY . .
```

¿Qué hace?

- Copia todo el código fuente al contenedor
- Primer `.`: Directorio actual en tu máquina
- Segundo `.`: Directorio actual en el contenedor (`/app`)

Configuración de Permisos

```
# =====  
# ETAPA 7: Permisos y scripts  
# =====  
RUN chmod +x /app/docker-entrypoint.sh
```

¿Por qué?

- Hace ejecutable el script de entrada
- Necesario para que Docker pueda ejecutar el script

Puerto de la Aplicación

```
# =====  
# ETAPA 8: Exposición de puerto  
# =====  
EXPOSE 8000
```

¿Qué hace?

- Documenta que la aplicación usa el puerto 8000
- **NO abre** el puerto (eso lo hace docker-compose)
- Es principalmente documentación

Comando de Inicio

```
# =====  
# ETAPA 9: Comando de inicio
```

```
# =====  
CMD ["/app/docker-entrypoint.sh"]
```

¿Qué hace?

- Define el comando que se ejecuta cuando inicia el contenedor
- Ejecuta el script `docker-entrypoint.sh`

docker-compose.yml Explicado

El **docker-compose.yml** define cómo orquestar múltiples contenedores.

Estructura General

```
version: "3.8" # Versión de Docker Compose  
  
services: # Definición de contenedores  
  web: # Contenedor Django  
  db: # Contenedor PostgreSQL  
  
volumes: # Volúmenes para persistencia  
networks: # Redes (opcional, se crea automáticamente)
```

Servicio Web (Django)

```
services:  
  web:  
    build: . # Construye imagen desde Dockerfile  
    ports:  
      - "8000:8000" # Mapea puerto host:contenedor  
    volumes:  
      - ./app # Monta código fuente  
      - static_files:/app/static  
      - media_files:/app/media  
    environment:  
      - DEBUG=True # Variables de entorno  
      -  
    DATABASE_URL=postgresql://ecodisseny:ecodisseny123@db:5432/ecodisseny_db  
    depends_on:  
      - db # Espera a que PostgreSQL esté listo  
    command: python manage.py runserver 0.0.0.0:8000
```

Explicación detallada:

Build y Imagen

```
build: .
```

- Construye la imagen usando el Dockerfile del directorio actual
- Equivale a: `docker build -t nombreproyecto_web .`

Mapeo de Puertos

```
ports:  
  - "8000:8000"
```

Concepto	Valor	Explicación
Puerto Host	8000	Puerto en tu VPS/máquina
Puerto Contenedor	8000	Puerto dentro del contenedor
Acceso	http://161.97.147.142:8000	Desde internet

Volúmenes del Servicio Web

```
volumes:  
  - ./app # Código fuente  
  - static_files:/app/static # Archivos estáticos  
  - media_files:/app/media # Archivos subidos
```

Tipos de volúmenes:

Tipo	Ejemplo	Función
Bind Mount	<code>./app</code>	Monta directorio del host
Named Volume	<code>static_files:/app/static</code>	Volumen gestionado por Docker

¿Por qué diferentes tipos?

- **Bind Mount (`./app`):**
 - ✔ Desarrollo: Cambios en código se reflejan inmediatamente
 - ✔ Depuración: Puedes editar archivos desde el host
- **Named Volume (`static_files`):**
 - ✔ Persistencia: Los datos sobreviven al reinicio del contenedor
 - ✔ Rendimiento: Optimizado por Docker
 - ✔ Backup: Fácil de respaldar

Variables de Entorno

```
environment:
  - DEBUG=True
  -
DATABASE_URL=postgresql://ecodisseny:ecodisseny123@db:5432/ecodisseny_db
```

Desglose de DATABASE_URL:

```
postgresql://usuario:contraseña@host:puerto/basedatos
      ↓           ↓           ↓           ↓           ↓           ↓
postgresql://ecodisseny:ecodisseny123@db:5432/ecodisseny_db
```

Componente	Valor	Explicación
protocolo	postgresql://	Tipo de base de datos
usuario	ecodisseny	Usuario de PostgreSQL
contraseña	ecodisseny123	Contraseña
host	db	Nombre del servicio Docker
puerto	5432	Puerto de PostgreSQL
base de datos	ecodisseny_db	Nombre de la BD

Dependencias

```
depends_on:
  - db
```

¿Qué hace?

- Django espera a que PostgreSQL esté **iniciado**
- **NOTA:** No espera a que esté **listo** para conexiones
- Por eso usamos `docker-entrypoint.sh` para esperar conexión

Servicio DB (PostgreSQL)

```
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_DB: ecodisseny_db
      POSTGRES_USER: ecodisseny
      POSTGRES_PASSWORD: ecodisseny123
    volumes:
```

```
- postgres_data:/var/lib/postgresql/data
ports:
- "5432:5432" # Solo para desarrollo
```

Explicación detallada:

Imagen Oficial

```
image: postgres:15
```

- Usa imagen oficial de PostgreSQL versión 15
- No necesita Dockerfile personalizado
- Imagen mantenida por PostgreSQL

Variables de PostgreSQL

```
environment:
  POSTGRES_DB: ecodisseny_db # Crea esta base de datos
  POSTGRES_USER: ecodisseny # Crea este usuario
  POSTGRES_PASSWORD: ecodisseny123 # Con esta contraseña
```

¿Qué hace PostgreSQL al iniciar?

1. 🗄️ Crea la base de datos **ecodisseny_db**
2. 👤 Crea el usuario **ecodisseny**
3. 🔑 Asigna la contraseña **ecodisseny123**
4. ✔️ Da permisos completos al usuario sobre la BD

Persistencia de Datos

```
volumes:
- postgres_data:/var/lib/postgresql/data
```

¿Por qué es importante?

- 💾 **Persistencia:** Los datos sobreviven al reinicio
- ↺ **Actualizaciones:** Los datos se mantienen al actualizar
- 📦 **Portabilidad:** El volumen se puede respaldar

Puerto de PostgreSQL

```
ports:
- "5432:5432"
```


Para desarrollo:

-  Puedes conectar herramientas como pgAdmin
-  Depuración desde herramientas externas

Para producción:

-  Se elimina por seguridad
-  Solo Django puede acceder a PostgreSQL

Definición de Volúmenes

```
volumes:
  postgres_data: # Datos de PostgreSQL
  static_files: # Archivos CSS, JS, imágenes estáticas
  media_files: # Archivos subidos por usuarios
```

¿Dónde se almacenan?

En Linux/VPS:

```
/var/lib/docker/volumes/
├── ecodisseny_dj_pg_postgres_data/
├── ecodisseny_dj_pg_static_files/
└── ecodisseny_dj_pg_media_files/
```

 Volúmenes y Persistencia

Tipos de Datos en la Aplicación

Tipo	Ejemplos	Persistencia	Volumen
Código Fuente	.py, .html, .css	Bind Mount	<code>./app</code>
Base de Datos	Usuarios, presupuestos	Named Volume	<code>postgres_data</code>
Archivos Estáticos	CSS, JS compilados	Named Volume	<code>static_files</code>
Media	PDFs, imágenes subidas	Named Volume	<code>media_files</code>

Flujo de Archivos Estáticos

 Desarrollo:
python manage.py collectstatic
↓

 Local: static/ →  Contenedor: /app/static/
↓
 Volumen: static_files (persistente)
↓
 Nginx: Sirve archivos directamente

Backup de Volúmenes

```
# Backup de base de datos
docker-compose exec db pg_dump -U ecodisseny ecodisseny_db > backup.sql

# Backup de volúmenes
docker run --rm -v ecodisseny_dj_pg_postgres_data:/data -v $(pwd):/backup
alpine tar czf /backup/postgres_backup.tar.gz -C /data .
```

Redes y Comunicación

Red Interna Docker

 Red Docker (automática)

-  web (Django) → IP: 172.20.0.2
-  db (PostgreSQL) → IP: 172.20.0.3
-  Comunicación: Por nombre de servicio

Resolución DNS Interna

Dentro de Docker Compose:

Desde	Hacia	URL
Django	PostgreSQL	db:5432
Django	Django	web:8000

¿Por qué funciona?

- Docker Compose crea una red automáticamente
- Cada servicio es accesible por su nombre
- No necesitas IPs específicas

Puertos Expuestos vs Internos

```
# PostgreSQL
ports:
- "5432:5432" # HOST:CONTENEDOR
```

Desarrollo:

- 🌐 **Externo:** localhost:5432 (desde tu máquina)
- 🐳 **Interno:** db:5432 (desde Django)

Producción (sin expose):

- ✖ **Externo:** No accesible
- ✓ **Interno:** db:5432 (solo Django)

⚙ Variables de Entorno

Jerarquía de Configuración

- 📁 **.env** (archivo) ← Prioridad más alta
- 🐳 **environment** (docker-compose)
- 🔧 **ENV** (Dockerfile)
- ⚙ **settings.py** (defaults) ← Prioridad más baja

Variables de Desarrollo vs Producción

Variable	Desarrollo	Producción	¿Por qué?
DEBUG	True	False	Seguridad
DATABASE_URL	db:5432	localhost:5432	Arquitectura
ALLOWED_HOSTS	*	app.arasmu.net	Seguridad
SECRET_KEY	Simple	Compleja	Seguridad

Archivo .env para Producción

```
# .env (en producción)
DEBUG=False
SECRET_KEY=tu_clave_super_secreta_de_64_caracteres
DATABASE_URL=postgresql://usuario:password@localhost:5432/ecodisseny_prod
ALLOWED_HOSTS=app.arasmu.net,161.97.147.142
```

🚀 Comandos Útiles

Gestión de Contenedores

```
# Construir e iniciar
docker-compose up -d
```

```
# Solo construir
docker-compose build

# Ver estado
docker-compose ps

# Ver logs
docker-compose logs -f web

# Parar servicios
docker-compose down

# Parar y eliminar volúmenes (⚠ PELIGROSO)
docker-compose down -v
```

Ejecutar Comandos Django

```
# Shell de Django
docker-compose exec web python manage.py shell

# Migraciones
docker-compose exec web python manage.py migrate

# Crear superusuario
docker-compose exec web python manage.py createsuperuser

# Collectstatic
docker-compose exec web python manage.py collectstatic

# Bash en el contenedor
docker-compose exec web bash
```

Gestión de Base de Datos

```
# Conectar a PostgreSQL
docker-compose exec db psql -U ecodisseny -d ecodisseny_db

# Backup de BD
docker-compose exec db pg_dump -U ecodisseny ecodisseny_db > backup.sql

# Restaurar BD
cat backup.sql | docker-compose exec -T db psql -U ecodisseny -d ecodisseny_db
```

Debugging y Monitoreo

```
# Ver recursos utilizados
docker stats

# Inspeccionar volúmenes
docker volume ls
docker volume inspect ecodisseny_dj_pg_postgres_data

# Ver redes
docker network ls
docker network inspect ecodisseny_dj_pg_default

# Logs específicos
docker-compose logs --tail=50 web
docker-compose logs --since="2025-08-21T18:30:00" db
```

Limpieza del Sistema

```
# Limpiar contenedores parados
docker container prune

# Limpiar imágenes no utilizadas
docker image prune

# Limpiar volúmenes no utilizados
docker volume prune

# Limpieza completa (⚠ CUIDADO)
docker system prune -a
```

Flujo Completo de Desarrollo

1. Desarrollo Local

```
# Iniciar desarrollo
docker-compose up -d

# Hacer cambios en código
# (se reflejan automáticamente por bind mount)

# Ejecutar migraciones si es necesario
docker-compose exec web python manage.py migrate

# Ver logs
docker-compose logs -f web
```

2. Testing

```
# Ejecutar tests
docker-compose exec web python manage.py test

# Test específico
docker-compose exec web python manage.py test carregahores.tests
```

3. Preparar para Producción

```
# Construir imagen optimizada
docker-compose -f docker-compose.prod.yml build

# Test en modo producción
docker-compose -f docker-compose.prod.yml up -d
```

4. Despliegue

```
# En el VPS
git pull origin docker
./deploy-vps.sh
```

🔍 Troubleshooting Común

Problema: Contenedor no inicia

```
# Ver logs detallados
docker-compose logs web

# Reconstruir imagen
docker-compose build --no-cache web

# Verificar Dockerfile
docker build -t test-image .
```

Problema: Base de datos no conecta

```
# Verificar que PostgreSQL está funcionando
docker-compose exec db pg_isready -U ecodisseny

# Test de conexión
docker-compose exec web python manage.py dbshell
```

```
# Verificar variables de entorno
docker-compose exec web env | grep DATABASE
```

Problema: Archivos estáticos no cargan

```
# Recolectar archivos estáticos
docker-compose exec web python manage.py collectstatic --noinput

# Verificar volumen
docker volume inspect ecodisseny_dj_pg_static_files

# Verificar permisos
docker-compose exec web ls -la /app/static/
```

Problema: Cambios no se reflejan

```
# Verificar bind mount
docker-compose exec web ls -la /app/

# Reiniciar solo Django
docker-compose restart web

# Verificar que el archivo cambió en el contenedor
docker-compose exec web cat manage.py
```

Recursos Adicionales

Documentación Oficial

-  [Docker Documentation](#)
-  [Docker Compose Documentation](#)
-  [Python Docker Best Practices](#)

Herramientas Útiles

- **Docker Desktop:** Interfaz gráfica para Docker
- **Portainer:** Panel web para gestionar Docker
- **Lazydocker:** TUI para Docker en terminal

Best Practices

1.  **Imágenes ligeras:** Usa `-slim` o `-alpine`
2.  **.dockerignore:** Excluye archivos innecesarios
3.  **Multi-stage builds:** Para imágenes de producción
4.  **Tags específicos:** No uses `latest` en producción

5.  **Volúmenes:** Para todos los datos persistentes
6.  **Secrets:** No hardcodees contraseñas
7.  **Health checks:** Para verificar que servicios están listos

¡Esta configuración Docker te permite desarrollar localmente y desplegar en producción con confianza!

